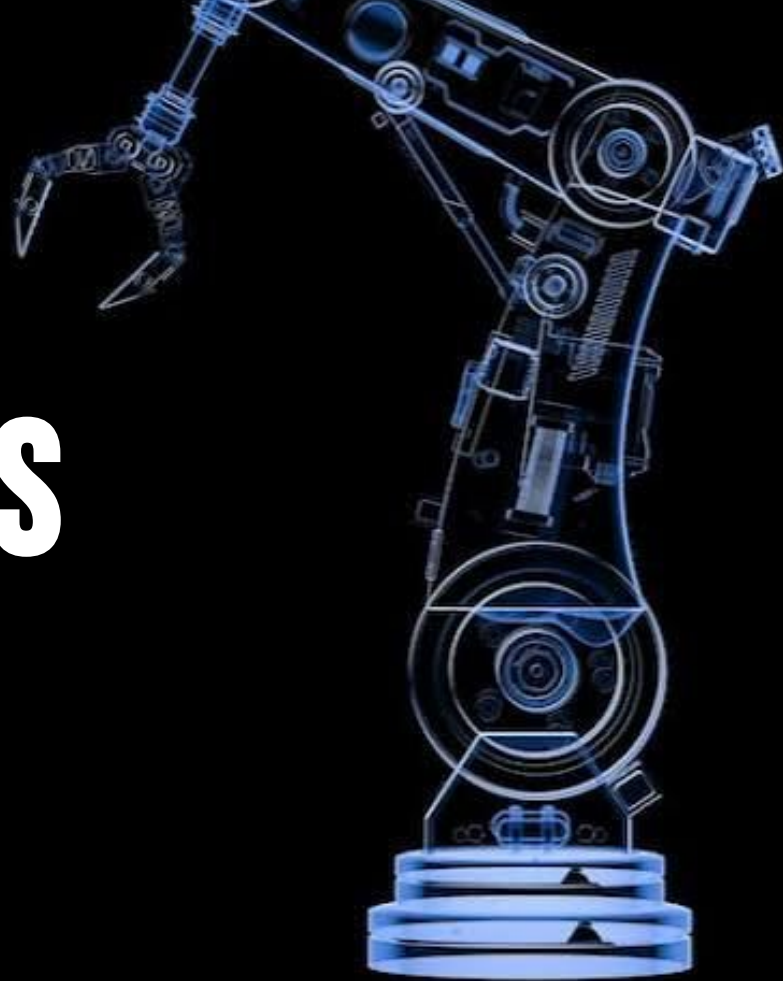


FOR FARMERS BY FARMERS

HEPHAESTUS

BY LILY, YVONNE, LEO, DIMITRIOS, ALISTAIR



01.

INTRODUCTION

The company

Hephaestus is the first company ever made, that produces machinery, specifically capable of sorting fruits.

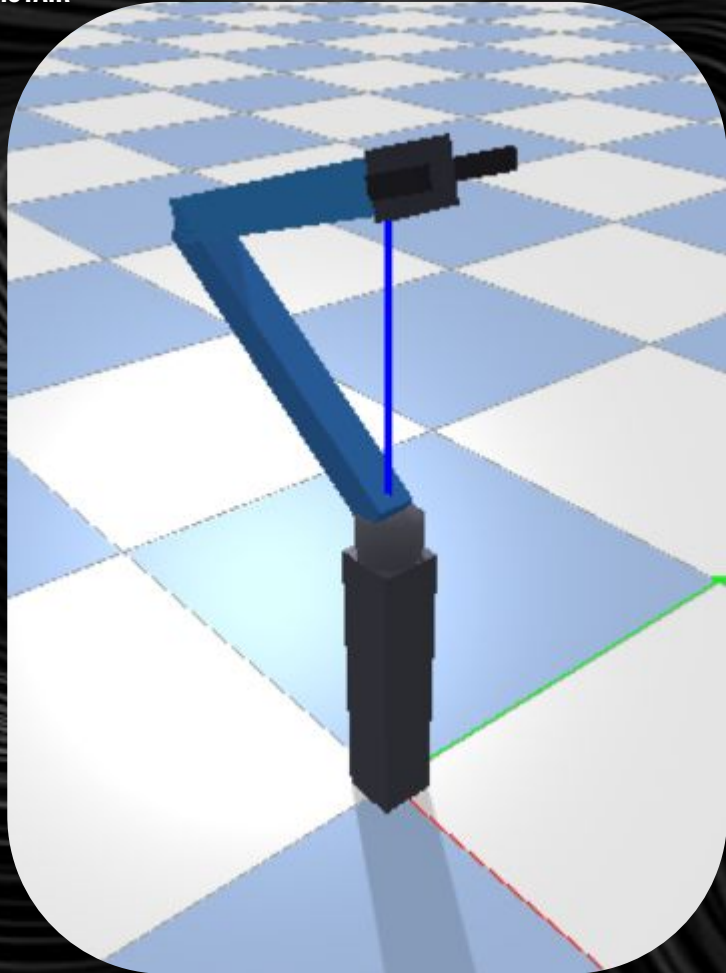
Problem

- Manual sorting
- Time-consuming
- High labor costs
- Inefficient process

Solution

- Autonomous vision
- Intelligent motion planning
- Accurate sorting
- Efficient automation





The Robotic Arm

Our robotic arm contains:

- A robust design with a strong foundation, that provides the required stability
- Precise end effector that is able to move in every direction and grab the fruits
- The ideal amount of joints and connectors for the sorting task

02.

CODE LOGIC

BRAINSTORMING

We chose fruit for the AI to pick up because the different colors and shape would be less complex for the AI. Allowing it to see different shapes and colors. Such as blue, green, red, orange, and yellow.



Threshold

- Converts an image into **black** and **white** based on a chosen color
- Pixels **above** the threshold become one color
- Pixels **below** the threshold become the other color
- Purpose: **Separate objects** from the background.

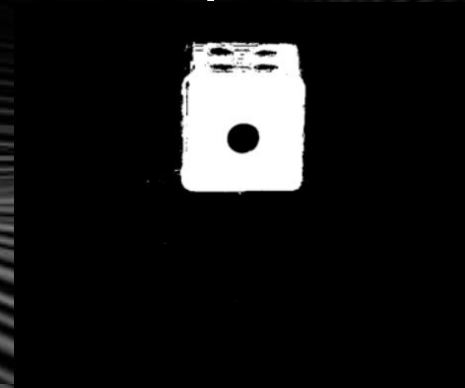
Masking

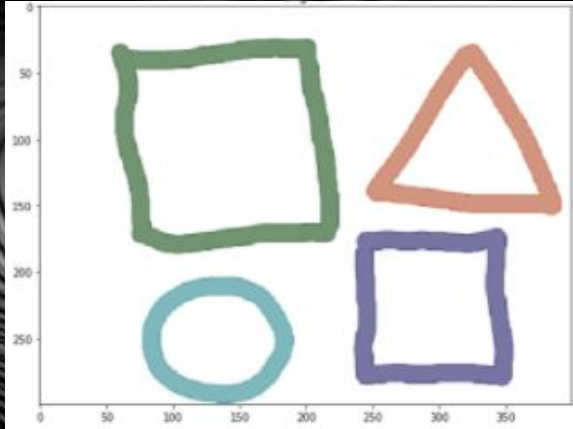
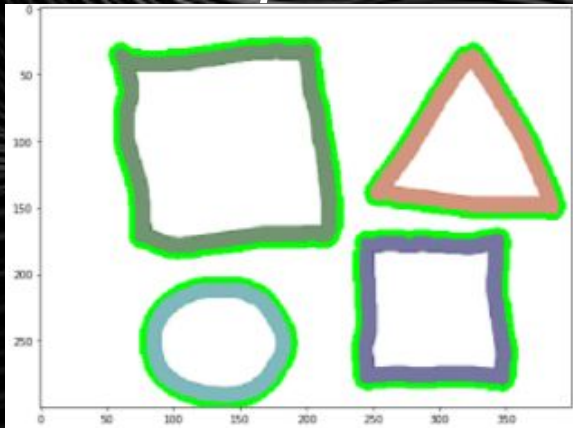
- **Keeps** the pixels you want and hides or **ignores** the rest, according to the threshold filter
- The mask is usually a black-and-white image:
 - **White** = keep this area.
 - **Black** = ignore this area.
- Purpose: **Focus processing** on the object or region of interest.

Before



After



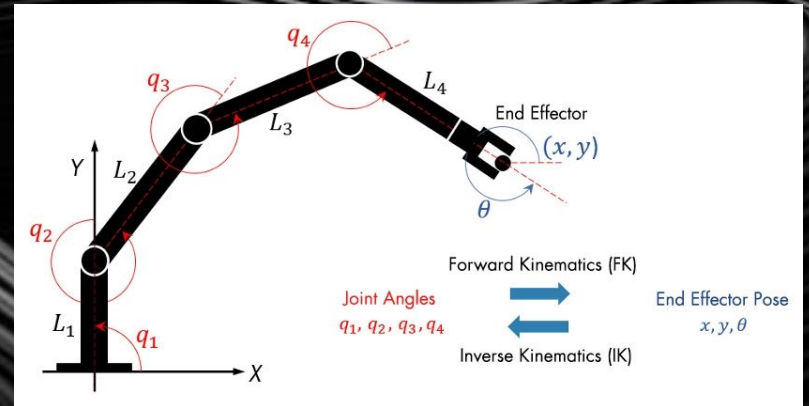
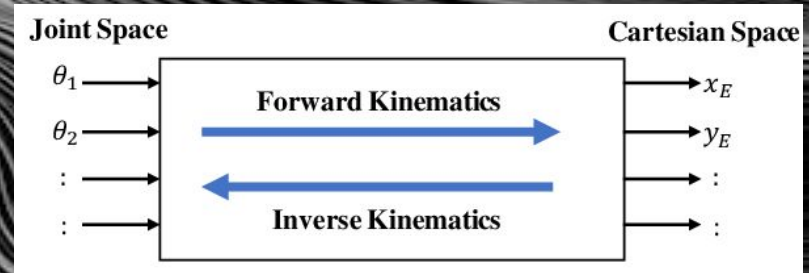
Before*After*

Contour

- **Contours** are the **boundaries** of objects in an image.
- The image is blurred, converted to HSV, and thresholded to separate the object from the background.
- The robot then uses a contour detection algorithm (such as OpenCV `findContours()`) to trace the object's boundary.
- From the contour, the robot can calculate:
 - **Center (centroid):** where the object is.
 - **Area:** how large the object appears.
 - **Perimeter:** the length of the object's outline.
 - **Bounding box:** the object's width, height, and position.
 - **Shape:** whether the object is circular, rectangular, triangular, etc.

Inverse Kinematics

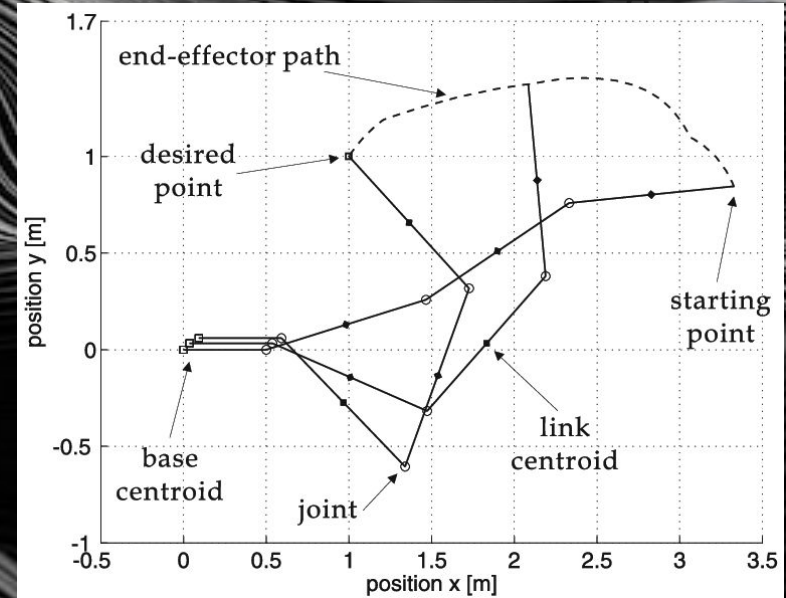
- **Inverse kinematics** calculates the **joint angles** a robot needs to move its end (gripper/tool) to a desired position.
- You give the robot the **target position** (e.g., x, y, z coordinates).
- The IK algorithm calculates **how much each joint should rotate**.
- **Purpose:** Move the robot's end-effector accurately to a specific location.
- Commonly used for **robot arms**, animation, and industrial robots.



Path planning

- **Cartesian** path planning **moves** the robot's end-effector (gripper) along a **straight or predefined path** in 3D space.
- You specify a series of positions (x, y, z), not just the final destination.
- The robot calculates the required joint movements for every point along the path using **inverse kinematics**.
- Purpose: Ensure the gripper follows a **smooth, accurate path** without sudden movements.
- Commonly used for **pick-and-place, welding, painting, and assembly tasks**.

what parts are



03.

CODING THE ROBOT ARM

The journey to coding the robotic arm

Libraries

```
import pybullet as p
import pybullet_data
import numpy as np
import cv2
import math
import time
```

Simulation setup

```
p.connect(p.GUI)
```

```
p.setAdditionalSearchPath(pybullet_data.  
getDataPath())
```

```
p.setGravity(0, 0, -9.8)
```

```
p.configureDebugVisualizer(p.COV_ENABLE  
GUI, 1)
```

```
p.loadURDF("plane.urdf")
```


Computer Vision

```
def scan_all():  
    rgb = snapshot()  
    hsv = cv2.cvtColor(rgb,  
cv2.COLOR_RGB2HSV)  
    found = []  
    for name, info in TYPES.items():  
        ...  
        found.append((name, (float(w[0]),  
float(w[1]))))  
    return found
```

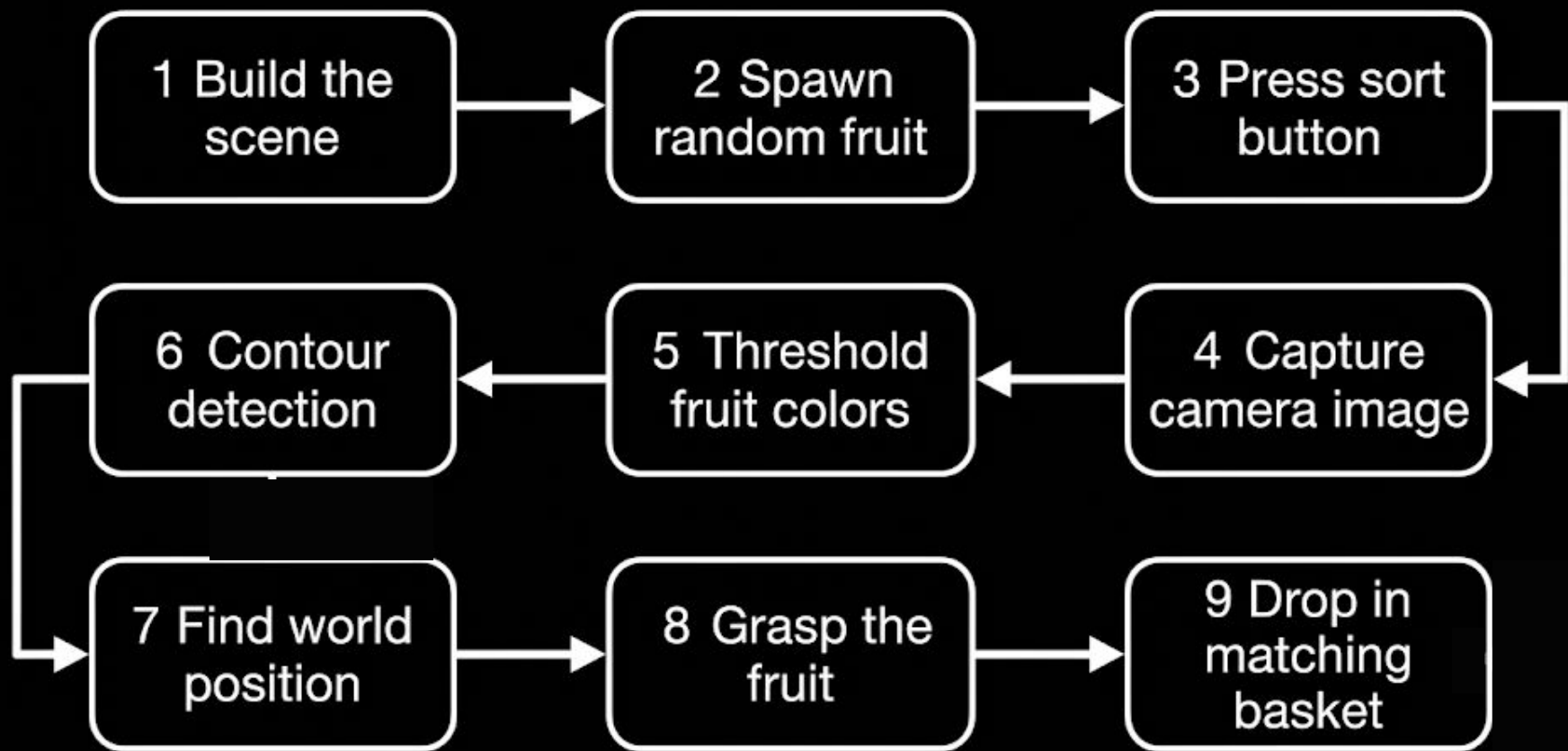
Robot motion (Inverse Kinematics)

```
def move_to(target, steps=900, tol=0.02):  
    sol = p.calculateInverseKinematics(  
        arm, PALM, target,  
        maxNumIterations=200,  
        residualThreshold=1e-4  
    )  
    for j, val in zip(ARM_JOINTS, sol[:3]):  
        p.setJointMotorControl2(  
            arm, j,  
            p.POSITION_CONTROL,  
            targetPosition=val,  
            force=350  
        )
```

Pick up and Place Sorting

```
def pick_and_sort(ftype, xy):  
    set_fingers(0.05)          # Open gripper  
    move_to([tx, ty, 0.60])    # Move above fruit  
    set_fingers(0.008)        # Close gripper  
    move_to([bx, by, 0.62])    # Move to basket  
    p.removeConstraint(grip)   # Release fruit  
    set_fingers(0.03)
```

Robotic Fruit Sorter How It Works



04.

DEMO

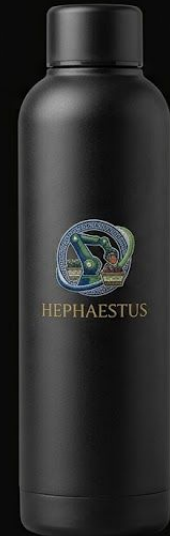
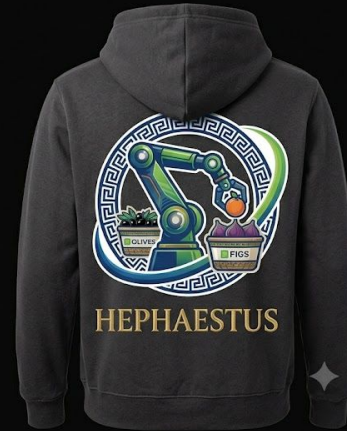
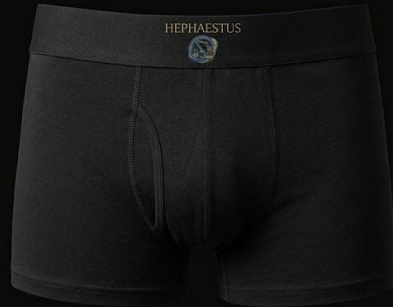
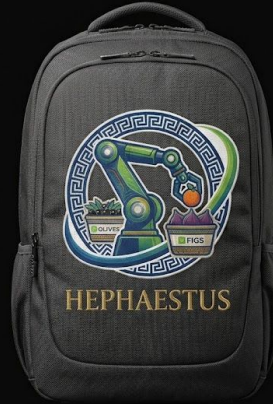


ARTIFICIAL INTELLIGENCE (AI)

ARTIFICIAL INTELLIGENCE (AI)



Our Merch



<https://github.com/leosekhri/Fruit-Sorting-Arm>



**THANK YOU
FOR YOUR
ATTENTION**

**By: Leo, Lily, Yvonne,
Alistair, Dimitrios**

The simulation Dimitrios' repository Leo's repository

